# Can you design a chip? Announcing the protocol emulator ASIC competition

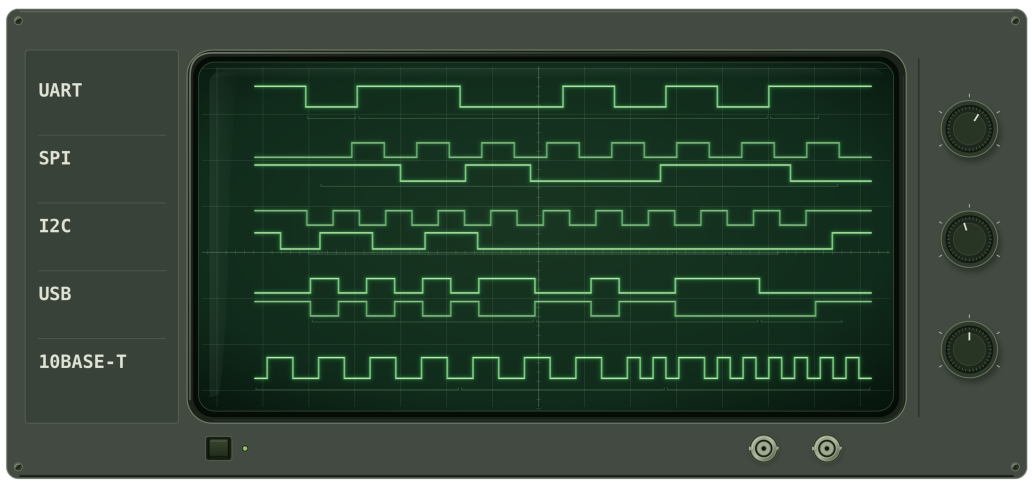
SEP 10, 2026 | 4 MIN READ



By: Benjamin Devlin



By: Anish Singhani




Last month, we asked you to reverse engineer a chip from nothing but its layout and teased a bigger challenge. Results and our favorite writeups are coming soon. In the meantime, here's our next challenge!

This time, you're designing the chip, and we'll pay to fabricate our favorite designs! We're particularly interested in projects with unique functionality, as well as those that demonstrate novel approaches to design and verification methodologies! Winners will receive a fabricated copy of their chip, mounted on a dev boards, so they can test their design in real silicon.

## The challenge

Design an **open-source, general-purpose protocol emulator ASIC**.

Hardware protocols like UART, SPI, and I2C are simple enough that people routinely "bit-bang" them: toggle pins from software with careful timing instead of using a dedicated peripheral. A protocol emulator is a small chip built to do exactly that: a tiny CPU with an instruction set designed for reading pins, writing pins, counting cycles, and hitting timing precisely enough that you can implement a real protocol in firmware rather than in fixed logic. Something like that is a useful tool for hardware debugging and reverse engineering, which is a good part of what we do.

The hard part is flexibility. The goal isn't to put a UART block, an SPI block, and an I2C block on one die and call it done. Your chip should be reprogrammable enough to support new protocols *after* fabrication, within its timing and I/O constraints. For inspiration, look at the PIO state machines on the RP2040 or the PRU cores on TI's Sitara parts, and consider what you'd do differently.

- Start with UART, SPI, and I2C.
- Stretch goals include low-speed USB and 10Mbit Ethernet.
- Other interesting protocols to consider: JTAG, SWD, PS/2, CAN bus
- If you have access to an FPGA, consider using it to test your RTL before the ASIC flow.
- Show us anything else your architecture makes possible that we haven't thought of.

At Jane Street, we use Hardcaml to generate the RTL for our FPGA and ASIC designs. We are excited to see the languages and verification techniques you use, including formal methods, random constrained tests, AI-assisted verification, and more. As AI-assisted chip design becomes more common, we believe verification will be an extremely important aspect of the ASIC design flow going forwards.

## The rules

- **Process:** We're targeting IHP's 130nm CMOS5L process through our friends at Tiny Tapeout. Start with the CMOS5L Verilog template, which takes you from RTL to GDS. Set the tile size in `info.yaml` to 6x4.
- **Area:** The current maximum area is 6x4 tiles per design. **We are working on the possibility of scaling up to 8x4 tiles (~30% more area). We'll update this page, as well as emailing everyone who has signed up if the larger tile size becomes available.**
- **Open source:** Your submission should be open source so others can use and build on it. Unlike the reverse-engineering puzzle, there's no need to keep your work hidden until the deadline, so feel free to build in public!
- **Teams:** This is a much bigger project than the puzzle, so we strongly recommend working in teams.
- **Deadline:** Submit your design by **January 18th, 2027**.
- **Prize:** We'll pay to tape out the most novel designs on a Tiny Tapeout shuttle. We're targeting the **March 2027 CMOS5L shuttle**, subject to the foundry schedule. Winners will receive chips and dev boards back after fabrication, so you can test your design in silicon.

## How much fits?

An 6x4 allocation is 24 tiles. At approximately 200um × 150um per tile, that's about 0.7 mm² of nominal tile area. As a rough estimate, budget for about 1K logic cells per tile. You may need to get creative to fit the functionality you want.

For instruction memory, SRAM can be more area-efficient than flip-flops. Tiny Tapeout has examples of SRAM running on this process node you can reference.

Run synthesis early, check the mapped cell area, and leave room for clock-tree buffers and routing. Then run the full place-and-route flow and check timing. A design that looks small enough after synthesis can still be difficult to route or too slow at your chosen clock frequency.

SUBSCRIBE TO EMAIL UPDATES

Email Address

Subscribe

SIGNALS & THREADS PODCAST

"At Jane Street, you can do whatever you want, as long as it's the right thing to do."

Listen to the latest episode

FEATURED TECH TALK

The Algorithm for Precision Medicine




Jane Street YouTube Channel

FEATURED READS



Using ASCII waveforms to test hardware designs



Finding memory leaks with Memtrace

TAGS

async, book, c, camlp4, code-review, comments, compiler, core, hackerschool, hg, incremental, internship, interviewing, machine-learning, ocaml, parallel-programming, performance, ppx, real-world-ocaml, registers, speed, ui

RSS

Get the RSS feed

JANE STREET OPEN SOURCE

From committing patches to the Linux kernel to releasing our own projects, we're always looking for ways to participate in the open source community.

JOIN OUR TEAM

## Getting started

If you've never taped out a chip before, the Tiny Tapeout documentation walks through the process end to end, and the tools are all free and open source. Start by getting a UART transmitter out of a pin. Then make it programmable.

**Sign-up** If you're interested, please fill out our sign-up form. We'll send updates about the tapeout template, deadlines, as well as providing the final submission link. Note that filling out the form is not a commitment to participating, it's just to receive updates!

We'll add a final submission form to this page closer to the deadline!

If you have questions along the way, reach out to asic-competition@janestreet.com.

## Hardware at Jane Street

The hardware team at Jane Street designs FPGAs and ASICs that run some of the fastest trading systems in the world. If designing a chip for fun sounds like your kind of thing, take a look at our hardware internships and full-time roles. You can also explore Hardcaml, our open-source OCaml hardware design libraries, or stay in touch.

*Ben is a hardware developer at Jane Street. Originally from New Zealand, with a Ph.D. in EE from the University of Tokyo, he is interested in the outdoors, sci-fi, cryptography, and pretty much anything tech related.*

*Anish joined Jane Street as an FPGA engineer in 2024 after graduating from Carnegie Mellon. He particularly enjoys building software tools to make hardware design more efficient.*

---

PREVIOUS POST NEXT POST

Jane Street

Where FP meets the real world.

If you're interested in working at a place where functional programming meets the real world, then apply for a job at Jane Street.

Learn more



© Copyright 2015-2026 Jane Street Group, LLC. All rights reserved. Services are provided in the U.S. by Jane Street Capital, LLC and Jane Street Execution Services, LLC, each of which is a SEC-registered broker dealer and member of FINRA (www.finra.org). Regulated activities are undertaken in Europe by Jane Street Financial Limited, an investment firm authorized and regulated by the U.K. Financial Conduct Authority, and Jane Street Netherlands B.V., an investment firm authorized and regulated by the Netherlands Authority for the Financial Markets (*Autoriteit Financiële Markten*), and in Hong Kong by Jane Street Hong Kong Limited, a regulated entity under the Hong Kong Securities and Futures Commission (CE No. BAL548). Each of these entities is a wholly owned subsidiary of Jane Street Group, LLC. This material is provided for informational purposes only and does not constitute an offer or solicitation for the purchase or sale of any security or other financial instrument. | Jane Street and the concentric circle mark are registered trademarks of Jane Street.

Privacy Cookies